



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO.
IIMAS

Calidad y Preproces. de Datos

Semestre 2025-1.

Profesores: Víctor Manuel Corza Vargas

Cinthia Rodríguez Maya

Practica 2

Integrantes:

- Aguilar Martínez Erick Yair
- Mendoza Hernández Carlos Emiliano
- Mendoza Sáenz de Buruaga Imanol
- Villalón Pineda Luis Enrique .

Practica 2

Sección 1, Carga de datos

Se importan las bibliotecas necesarias para la primera sección

```
In [1]: import pandas as pd
import matplotlib.pyplot as plt
from sklearn.base import BaseEstimator, TransformerMixin
from sklearn.pipeline import Pipeline
import missingno as msno
from itables import show
import numpy as np
import uuid
from IPython.display import display, HTML
```

```
In [2]: df=pd.read_csv('./winequality-white.csv')
print("Información del dataset")
df.info()
```

```
Información del dataset
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5338 entries, 0 to 5337
Data columns (total 12 columns):
#   Column                Non-Null Count  Dtype
---  -
0   fixed acidity          4809 non-null   float64
1   volatile acidity       4807 non-null   float64
2   citric acid            4794 non-null   float64
3   residual sugar         4794 non-null   float64
4   chlorides              4806 non-null   float64
5   free sulfur dioxide     4807 non-null   float64
6   total sulfur dioxide    4789 non-null   float64
7   density                4794 non-null   float64
8   pH                    4809 non-null   float64
9   sulphates              4803 non-null   float64
10  alcohol                4801 non-null   float64
11  quality                4799 non-null   float64
dtypes: float64(12)
memory usage: 500.6 KB
```

Imprimimos los 10 primeros registros para darnos una idea del data frame con .head()

```
In [3]: df.head(10)
```

Out[3]:

	fixed acidity	volatile acidity	citric acid	residual sugar	chlorides	free sulfur dioxide	total sulfur dioxide	density	pH	sulphat
0	7.0	0.27	0.36	20.7	0.045	45.0	170.0	1.0010	3.00	0.
1	6.3	0.30	0.34	1.6	0.049	NaN	NaN	0.9940	3.30	Na
2	8.1	NaN	0.40	6.9	0.050	NaN	97.0	0.9951	3.26	Na
3	7.2	NaN	0.32	8.5	0.058	47.0	186.0	0.9956	3.19	0.
4	7.2	0.23	NaN	8.5	0.058	NaN	186.0	0.9956	3.19	Na
5	8.1	0.28	0.40	6.9	0.050	30.0	NaN	0.9951	3.26	0.
6	6.2	0.32	0.16	7.0	0.045	30.0	136.0	0.9949	3.18	0.
7	7.0	0.27	0.36	20.7	0.045	45.0	170.0	1.0010	3.00	0.
8	NaN	0.30	0.34	1.6	0.049	14.0	132.0	0.9940	3.30	0.
9	8.1	0.22	0.43	1.5	0.044	28.0	129.0	0.9938	3.22	0.

In [4]: `df.shape`

Out[4]: (5338, 12)

1.a) Porcentaje de datos faltantes por columna

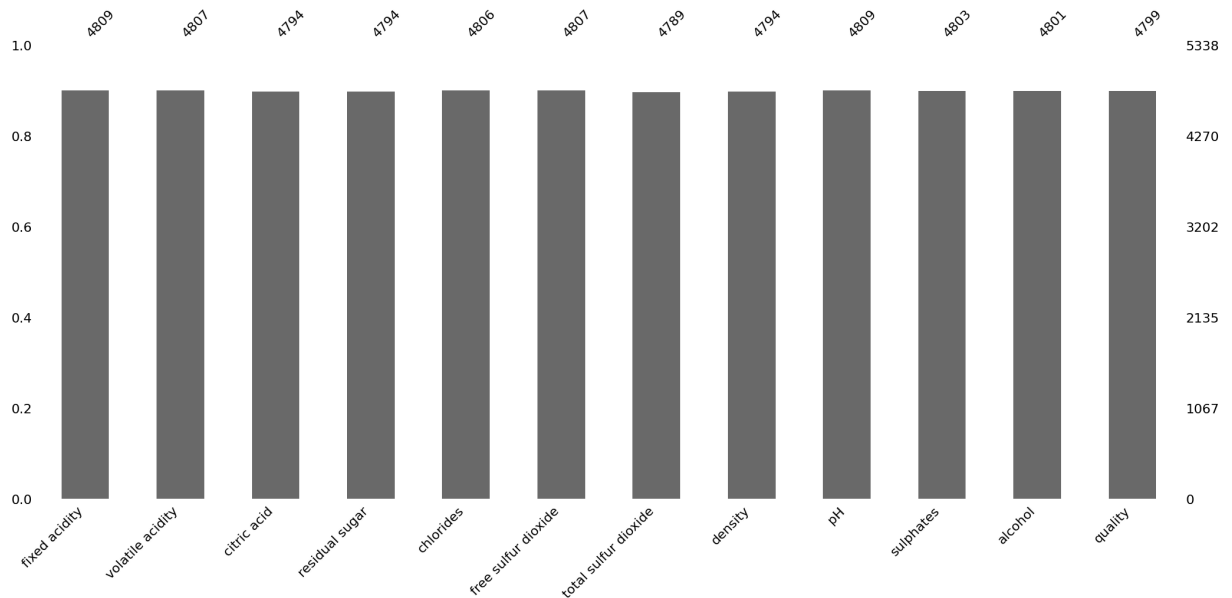
In [5]: `faltante = (df.isnull().sum() / len(df)) * 100`
`faltante`

Out[5]:

fixed acidity	9.910079
volatile acidity	9.947546
citric acid	10.191083
residual sugar	10.191083
chlorides	9.966280
free sulfur dioxide	9.947546
total sulfur dioxide	10.284751
density	10.191083
pH	9.910079
sulphates	10.022480
alcohol	10.059948
quality	10.097415

dtype: float64

In [6]: `msno.bar(df)`
`plt.show()`



Explicación y ejemplo

`df.isnull()` devuelve un data frame del mismo tamaño pero con valores booleanos de True o False si hay valor nulo o no

```
In [7]: df.isnull()
```

```
Out[7]:
```

	fixed acidity	volatile acidity	citric acid	residual sugar	chlorides	free sulfur dioxide	total sulfur dioxide	density	pH	quality
0	False	False	False	False	False	False	False	False	False	
1	False	False	False	False	False	True	True	False	False	
2	False	True	False	False	False	True	False	False	False	
3	False	True	False	False	False	False	False	False	False	
4	False	False	True	False	False	True	False	False	False	
...	...	...	...	...	...	...	...	...	...	
5333	False	False	False	False	False	False	True	False	False	
5334	False	False	False	False	True	False	False	True	False	
5335	False	False	False	False	False	False	False	False	False	
5336	False	False	False	False	False	False	False	False	False	
5337	False	False	False	False	False	False	False	True	False	

5338 rows × 12 columns

df.isnull().sum() devuelve la suma por columnas de valores faltantes

```
In [8]: df.isnull().sum()
```

```
Out[8]: fixed acidity      529
volatile acidity    531
citric acid         544
residual sugar      544
chlorides           532
free sulfur dioxide 531
total sulfur dioxide 549
density            544
pH                 529
sulphates          535
alcohol            537
quality            539
dtype: int64
```

```
In [9]: len(df) #Devuelve cuantos registros hay
```

```
Out[9]: 5338
```

$(df.isnull().sum() / len(df)) * 100$ es simplemente la formula de valores faltantes entre el total multiplicado por 100 para obtener un porcentaje

1.b) Porcentaje de datos duplicados

```
In [10]: porcentaje_duplicados = (df.duplicated().sum() / len(df)) * 100
print(f"Porcentaje de filas duplicadas: {porcentaje_duplicados: }%")
```

Porcentaje de filas duplicadas: 10.434619707755715%

Explicación y ejemplo

df.duplicated() devuelve una tabla de valores booleanos si los registros estan repetidos o no

```
In [11]: df.duplicated()
```

```
Out[11]: 0      False
1      False
2      False
3      False
4      False
...
5333   True
5334   True
5335   True
5336   True
5337   True
Length: 5338, dtype: bool
```

```
In [12]: df.duplicated().sum() #devuelve el numero total de filas duplicadas
```

```
Out[12]: np.int64(557)
```

```
In [13]: (df.duplicated().sum() / len(df)) * 100 #Calculo del porcentaje con formula
```

```
Out[13]: np.float64(10.434619707755715)
```

1.c) Porcentaje de registros completos vs porcentaje de registros incompletos en todo el dataset

```
In [14]: registros_completos = df.dropna().shape[0] # Filas sin valores faltantes
registros_incompletos = len(df) - registros_completos # Filas con al menos

# Calcular los porcentajes
porcentaje_completos = (registros_completos / len(df)) * 100
porcentaje_incompletos = (registros_incompletos / len(df)) * 100

# Mostrar los resultados
print(f"Porcentaje de registros completos: {porcentaje_completos: }%")
print(f"Porcentaje de registros incompletos: {porcentaje_incompletos: }%")
```

Porcentaje de registros completos: 27.632071937055073%

Porcentaje de registros incompletos: 72.36792806294491%

Explicación y ejemplo

df.dropna() elimina registros en los que al menos haya un valor faltante

```
In [15]: df.dropna()
```

Out[15]:

	fixed acidity	volatile acidity	citric acid	residual sugar	chlorides	free sulfur dioxide	total sulfur dioxide	density	pH	sulp
0	7.0	0.27	0.36	20.70	0.045	45.0	170.0	1.00100	3.00	
9	8.1	0.22	0.43	1.50	0.044	28.0	129.0	0.99380	3.22	
14	8.3	0.42	0.62	19.25	0.040	41.0	172.0	1.00020	2.98	
27	7.0	0.28	0.39	8.70	0.051	32.0	141.0	0.99610	3.38	
30	8.5	0.24	0.39	10.40	0.044	20.0	142.0	0.99740	3.20	
...	...	...	...	...	...	...	...	...	...	...
5326	6.1	0.36	0.26	8.15	0.035	14.0	88.0	0.99031	3.06	
5328	8.1	0.27	0.33	1.30	0.045	26.0	100.0	0.99066	2.98	
5329	6.5	0.51	0.25	1.70	0.048	39.0	177.0	0.99212	3.28	
5335	7.4	0.25	0.36	13.20	0.067	53.0	178.0	0.99760	3.01	
5336	5.7	0.28	0.36	1.80	0.041	38.0	90.0	0.99002	3.27	

1475 rows × 12 columns

df.dropna().shape[0] devuelve el numero de registros despues de eliminar registros con datos faltantes

```
In [16]: df.dropna().shape[0]
```

Out[16]: 1475

len(df) - registros_completos devuelve un numero, la longitud del data frame menos los que estan completos da el numero de registros incompletos

```
In [17]: len(df) - registros_completos
```

Out[17]: 3863

Se calcula con la formula ya explicada el porcentaje requerido

```
In [18]: (registros_completos / len(df)) * 100  
(registros_incompletos / len(df)) * 100
```

Out[18]: 72.36792806294491

Sección 2

Se eligió el eje de (**Salud**)

2.1 Selección de framework

Six Sigma

Six sigma utiliza herramientas estadísticas para la caracterización y el estudio de los procesos, de ahí el nombre de la herramienta, ya que sigma es la desviación típica que da una idea de la variabilidad en un proceso, y el objetivo de la metodología six sigma es reducir esta, de modo que el proceso se encuentre siempre dentro de los límites establecidos.

Six Sigma se centra en la reducción de defectos y mejora continua a través de la metodología DMAIC (Por sus siglas en inglés: *Define - Measure - Analyze - Improve - Control*).

- **Definir:** En el sector salud, es crucial definir claramente qué se considera "datos de calidad". Esto incluye identificar los requisitos de los stakeholders (médicos, enfermeras, administradores, pacientes) y establecer métricas de calidad específicas, como la precisión de los diagnósticos, la completitud de los historiales médicos o la consistencia en los registros de medicamentos.
- **Medir:** Medir la calidad de los datos es esencial en salud porque los errores pueden tener consecuencias graves, como diagnósticos incorrectos o tratamientos inadecuados. Se deben utilizar métricas como:
 - Precisión: ¿Los datos reflejan la realidad del paciente?
 - Consistencia: ¿Los datos son coherentes entre diferentes sistemas (historial clínico, farmacia, laboratorio)?
 - Completitud: ¿Faltan datos críticos en los registros médicos?
 - Oportunidad: ¿Los datos están disponibles y actualizados en tiempo real?
- **Analizar:** En salud, es fundamental identificar las causas raíz de los problemas de calidad de los datos, ya que estos pueden estar relacionados con procesos manuales, falta de capacitación del personal, integración deficiente de sistemas o errores humanos.
- **Mejorar:** Las soluciones para mejorar la calidad de los datos en salud deben ser robustas y enfocadas en la seguridad del paciente. Esto puede incluir la automatización de procesos, la implementación de controles de calidad, la capacitación del personal y la actualización de sistemas de información.
- **Controlar:** En salud, es esencial mantener la calidad de los datos a largo plazo. Esto implica:
 - Monitoreo continuo de métricas de calidad.
 - Auditorías periódicas de los registros médicos.
 - Establecimiento de políticas y procedimientos claros para la gestión de datos.

Esta metodología, fundamentalmente, se aplica para establecer la uniformidad en los procesos, en mejorar y dar soluciones a problemas complejos por medio de herramientas de control y reducción de variaciones en los procesos de más alto

desempeño, lo cual es esencial en el sector salud, donde la precisión y la oportunidad de los datos pueden impactar directamente en la vida de los pacientes. Además, Six Sigma permite monitorear y optimizar la calidad de los datos de manera estructurada.

Al aplicar Six Sigma en la gestión de datos en salud, se pueden minimizar errores en historiales médicos, mejorar la consistencia entre diferentes sistemas y asegurar que la información sea precisa y accesible cuando se necesite.

2.2 Ejes del modelo seleccionados

Dado que el sector salud es altamente regulado y sensible, consideraríamos todos los ejes de Six Sigma, pero con diferentes niveles de prioridad según las cuatro dimensiones clave de calidad de datos (*Accuracy, Timeliness, Consistency, Completeness*).

DIMENSION	Prioridad	Justificación
Precisión (Accuracy)	Muy alta	Un dato inexacto (por ejemplo, una dosis incorrecta de medicamento o un diagnóstico erróneo) puede poner en riesgo la vida de los pacientes. Es fundamental garantizar que los datos sean correctos y estén validados con estándares médicos.
Consistencia (Consistency)	Alta	Los registros médicos pueden provenir de múltiples fuentes (hospitales, laboratorios, farmacias). Asegurar que no haya discrepancias entre bases de datos es clave para evitar errores en tratamientos y diagnósticos.
Compleitud (Completeness)	Media-Alta	Un expediente clínico incompleto puede afectar la calidad del tratamiento. Es importante asegurar que los datos esenciales estén siempre registrados, aunque se pueden tolerar ciertas ausencias en información menos crítica.
Oportunidad (Timeliness)	Muy alta	En emergencias médicas, el acceso inmediato a datos precisos puede ser la diferencia entre la vida y la muerte. Es crucial minimizar los retrasos en la disponibilidad de información.

Se priorizaría mayormente la Precisión y la Oportunidad, ya que los errores en datos clínicos pueden ser fatales si no se detectan a tiempo. De igual manera, la Consistencia y la Compleitud son fundamentales para garantizar que los datos sean útiles y confiables en el largo plazo, especialmente en el sector salud, donde la continuidad de la atención y el seguimiento de los pacientes son críticos.

2.3 Modelo de clasificación para evaluar la calidad de datos

Para evaluar la calidad de los datos en el sector salud bajo el enfoque Six Sigma, proponemos el siguiente modelo de clasificación:

Ponderación de Dimensiones en el Ranking

Dimensión	Peso (%)	Justificación
Precisión (Accuracy)	40%	Un error en los datos médicos puede ser mortal. La exactitud en diagnósticos, tratamientos y medicamentos es prioritaria.
Oportunidad (Timeliness)	30%	En emergencias, los datos deben estar disponibles de inmediato. Un retraso puede afectar decisiones críticas.
Consistencia (Consistency)	20%	Evitar discrepancias entre registros es clave para garantizar tratamientos adecuados y continuidad del cuidado.
Compleitud (Completeness)	10%	Aunque es importante, se pueden manejar ciertos datos faltantes sin afectar directamente la seguridad del paciente.

Dado que en el sector salud la precisión y la oportunidad son críticas, estas dimensiones tendrán un mayor peso en la evaluación. La consistencia es fundamental para evitar errores en tratamientos y diagnósticos, mientras que la completitud, aunque importante, puede ser menos crítica en ciertos contextos.

Métricas Específicas por Dimensión

1. Precisión (Accuracy)

Métrica	Fórmula	Escala de Puntuación (0-10)
Porcentaje de coincidencia con fuentes confiables	$\frac{\text{Registros correctos}}{\text{Total de registros evaluados}} \times 100$	$10 \times \frac{\text{Coincidencia (\%)} }{100}$

2. Consistencia (Consistency)

Métrica	Fórmula	Escala de Puntuación (0-10)
Porcentaje de discrepancia	$\frac{\text{Registros inconsistentes}}{\text{Total de registros}} \times 100$	$10 - \left(10 \times \frac{\text{Discrepancias (\%)} }{100}\right)$

3. Compleitud (Completeness)

Métrica	Fórmula	Escala de Puntuación (0-10)
Porcentaje de registros llenos	$\frac{\text{Registros completos}}{\text{Total de registros}} \times 100$	$10 \times \frac{\text{Compleitud (\%)} }{100}$

4. Oportunidad (Timeliness)

Métrica	Fórmula	Escala de Puntuación (0-10)
Tiempo medio de actualización de datos	Promedio de tiempo entre evento y actualización (en minutos, horas o días). $10 - \left(\frac{\text{Tiempo medio de actualización}}{\text{Límite permitido}} \right) \times 10$	

Escala de Evaluación

Cada dimensión se evaluará en una escala de **0 a 10 puntos** y se multiplicará por su peso para obtener un puntaje final de **0 a 10** para el dataset.

La puntuación final es:

$$\text{Puntaje Final} = (P_{\text{Precisión}} \times 0.4) + (P_{\text{Oportunidad}} \times 0.3) + (P_{\text{Consistencia}} \times 0.2) + (P_{\text{C}}$$

Puntaje (0-10)	Clasificación	Interpretación
9.0 - 10	Óptima	Datos de alta calidad, confiables y listos para su uso clínico.
7.0 - 8.9	Buena	Datos con pequeños errores o retrasos, pero sin impacto crítico en la atención médica.
5.0 - 6.9	Regular	Datos con inconsistencias o faltantes que pueden afectar la toma de decisiones.
3.0 - 4.9	Deficiente	Datos con errores frecuentes, incompletos o no disponibles a tiempo.
0 - 2.9	Crítica	Datos inservibles para la atención médica. Urge intervención.

Ejemplo de Evaluación de un Dataset Médico

Supongamos que evaluamos un dataset de historias clínicas electrónicas en un hospital.

Dimensión	Puntaje (0-10)	Peso (%)	Puntaje Ponderado
Precisión	9.5	40%	3.8
Oportunidad	8.0	30%	2.4
Consistencia	7.5	20%	1.5
Compleitud	6.0	10%	0.6
Total	8.3	100%	8.3

Clasificación: "Buena" (7.0 - 8.9) → Datos adecuados pero con margen de mejora.

2.4 Mitigación de problemas

Problemas Identificados y Acciones de Mejora

Dimensión	Problema Detectado	Acciones Correctivas	Acciones Preventivas
Precisión (Accuracy)	Datos incorrectos en diagnósticos, dosis de medicamentos o antecedentes médicos.	♦ Implementar validaciones automáticas de datos con reglas médicas predefinidas. ♦ Realizar auditorías periódicas con muestreo aleatorio para corregir datos erróneos.	♦ Integrar un sistema de validación en tiempo real que compare datos ingresados con fuentes confiables (ej. terminologías médicas estándar como SNOMED-CT). ♦ Capacitación continua al personal en la captura de datos clínicos.
Consistencia (Consistency)	Discrepancias en los datos de un mismo paciente en distintos sistemas (ej. hospital, laboratorio, farmacia).	♦ Desarrollar procesos de conciliación automática entre bases de datos. ♦ Implementar un ETL para limpieza y consolidación de datos.	♦ Uso de identificadores únicos para pacientes (ej. número de expediente centralizado). ♦ Establecer estándares de interoperabilidad con proveedores externos.

Dimensión	Problema Detectado	Acciones Correctivas	Acciones Preventivas
Complejidad (Completeness)	Expedientes clínicos con datos faltantes (ej. alergias, antecedentes familiares).	- Alertas automáticas cuando se detectan campos críticos vacíos. - Diseño de formularios electrónicos obligatorios en el sistema de historia clínica.	- Políticas de captura obligatoria para campos esenciales. - Capacitación del personal sobre la importancia de completar los datos.
Oportunidad (Timeliness)	Datos desactualizados o retraso en la actualización de resultados médicos.	- Implementar procesos de actualización automatizada desde sistemas de laboratorio y farmacia. - Definir Acuerdos de Nivel de Servicio para tiempos máximos de actualización.	- Sistemas en la nube con acceso en tiempo real a datos clínicos. - Sensores IoT para monitoreo continuo de signos vitales y actualización inmediata.

Evaluación Continua: Dashboard de Calidad de Datos

Para monitorear la mejora en la calidad de datos, se puede implementar un dashboard con indicadores de desempeño, tales como:

Precisión: % de registros sin errores vs. total de registros.

Consistencia: % de discrepancias detectadas en bases de datos.

Complejidad: % de expedientes con campos obligatorios llenos.

Oportunidad: Tiempo promedio de actualización de datos críticos.

```
In [19]: css = HTML('''
<style>
.dt-container {
    font-size: 28px;
}
</style>
''')
display(css)
def showMetric(title, value):
    display(HTML(f'''
<div style="
    font-size: 20px;
    color: #333;
    padding: 10px;
    border: 1px solid #ccc;
    background-color: #f9f9f9;
    width: 100%;
    text-align: center;
    margin-bottom: 10px;
">{title}</div>
{value}
'''))
```

```

        border-radius: 5px;
        margin: 5px 0;
    ">
        <strong>{title}</strong> {value}
    </div>
    """)
def showTitle(t):
    display(HTML(f"""
        <div style="
            box-shadow: 0 4px 8px rgba(0,0,0,0.1);
            border-radius: 5px;
            margin: 20px 0;
            padding: 10px;
            text-align: center;
            background-color: #eee;
        ">
            <h1 style="
                font-family: 'Arial', sans-serif;
                color: #333;
                font-size: 1.5em;
                margin: 0;
            ">{t}</h1>
        </div>
    """))
class CsvReader(BaseEstimator, TransformerMixin):
    def __init__(self, path):
        self.path = path

    def fit(self, X, y=None):
        return self

    def transform(self, _):
        return pd.read_csv(self.path)

class ErrorFingerTransformer(BaseEstimator, TransformerMixin):
    def __init__(self, columns, p, seed=42):
        self.columns = columns if isinstance(columns, list) else [columns]
        self.p = p
        self.seed = seed

    def fit(self, X, y=None):
        return self

    def transform(self, X):
        new_df = X.copy()
        n_rows = len(new_df)
        m_cols = len(self.columns)
        np.random.seed(self.seed)
        mustBeChange = np.random.binomial(1, self.p, size=(n_rows, m_cols))
        for i in range(n_rows):
            for j in range(m_cols):
                if mustBeChange[i, j]:
                    value = new_df[self.columns[j]].iloc[i]
                    value_str = str(value)
                    value_str += np.random.choice(['1', '2', '3', '4', '5', '6', '7', '8', '9'])
                    new_val = new_df[self.columns[j]].dtype.type(value_str)

```

```

        new_df.iloc[i, new_df.columns.get_loc(self.columns[j])]
    return new_df

class DuplicateRecordsTransformer(BaseEstimator, TransformerMixin):
    def __init__(self, n_duplicates, columns=None, random_state=42):
        self.n_duplicates = n_duplicates
        self.columns = columns
        self.random_state = random_state

    def fit(self, X, y=None):
        return self

    def transform(self, X):
        new_df = X.copy()
        np.random.seed(self.random_state)
        random_indices = np.random.choice(len(X), self.n_duplicates, replace=True)
        duplicates = X.iloc[random_indices].copy()
        result_df = pd.concat([new_df, duplicates], ignore_index=True)
        return result_df

class MissingValuesTransformer(BaseEstimator, TransformerMixin):
    def __init__(self, columns, p, seed=42):
        self.columns = columns if isinstance(columns, list) else [columns]
        self.p = p
        self.seed = seed

    def fit(self, X, y=None):
        return self

    def transform(self, X):
        new_df = X.copy()
        i = 0
        for col in self.columns:
            if col in new_df.columns:
                np.random.seed(self.seed+i)
                mask = np.random.rand(len(new_df)) < self.p
                new_df.loc[mask, col] = np.nan
                i += 1
        return new_df

class DeleteRecordsTransformer(BaseEstimator, TransformerMixin):
    def __init__(self, p, seed=42):
        self.p = p
        self.seed = seed

    def fit(self, X, y=None):
        return self

    def transform(self, X):
        new_df = X.copy()
        n_rows = len(new_df)
        np.random.seed(self.seed)
        mustBeChange = np.random.binomial(1, self.p, size=n_rows)
        return new_df[~mustBeChange.astype(bool)]

class GenerateRandomUUID(BaseEstimator, TransformerMixin):

```

```

def __init__(self, column, random_state=42):
    self.column = column
    self.n_uuids = None
    self.random_state = random_state

def fit(self, X, y=None):
    self.n_uuids = len(X)
    return self

def transform(self, X):
    new_df = X.copy()
    np.random.seed(self.random_state)
    uuids = [uuid.uuid4().hex for _ in range(self.n_uuids)]
    new_df[self.column] = uuids
    return new_df

class CastTypeTransforme(BaseEstimator, TransformerMixin):
    def __init__(self, pairColumnType):
        self.pairColumnType = pairColumnType

    def fit(self, X, y=None):
        return self

    def transform(self, X):
        new_df = X.copy()
        for col, dtype in self.pairColumnType.items():
            if col in new_df.columns:
                new_df[col] = new_df[col].apply(lambda x: dtype(x) if pd.not
        return new_df

class AgregarInconsistencia(BaseEstimator, TransformerMixin):
    def __init__(self, id_column, columns, noise_factor=0.05, seed=42, p=0.9,
        """
        id_column: str
            The column used to identify duplicate groups.
        columns: list of str
            List of columns where variations will be added on duplicate groups.
        noise_factor: float
            Factor to scale the noise for numeric columns.
        seed: int
            Random seed.
        """
        self.id_column = id_column
        self.columns = columns if isinstance(columns, list) else [columns]
        self.noise_factor = noise_factor
        self.p = p
        self.p2 = p2
        self.seed = seed

    def fit(self, X, y=None):
        return self

    def transform(self, X):
        new_df = X.copy()
        np.random.seed(self.seed)
        groups = new_df.groupby(self.id_column)

```



```

        for _, group_df in groups:
            if len(group_df) > 1:
                for idx in group_df.index:
                    if np.random.binomial(1, self.p, 1)[0] == 0:
                        continue
                    mustBeChange = np.random.binomial(1, self.p2, len(self.columns))
                    for col, bitChange in zip(self.columns, mustBeChange):
                        if not bitChange:
                            continue
                        original_val = new_df.at[idx, col]
                        if pd.api.types.is_numeric_dtype(new_df[col]):
                            noise = original_val * np.random.uniform(-self.r, self.r)
                            new_val = original_val + noise
                            new_val = new_df[col].dtype.type(new_val)
                            new_df.at[idx, col] = new_val

    return new_df

class GenerateStartDate(BaseEstimator, TransformerMixin):
    def __init__(self, start_date, end_date, date_column):
        self.start_date = start_date
        self.end_date = end_date
        self.date_column = date_column

    def fit(self, X, y=None):
        return self

    def transform(self, X):
        new_df = X.copy()
        new_df[self.date_column] = pd.date_range(start=self.start_date, end=self.end_date, freq='D')
        return new_df

class RandomDateOffsetTransformer(BaseEstimator, TransformerMixin):
    def __init__(self, date_column, new_column, max_seconds, seed=42):
        self.date_column = date_column
        self.new_column = new_column
        self.max_seconds = max_seconds
        self.seed = seed

    def fit(self, X, y=None):
        return self

    def transform(self, X):
        new_df = X.copy()
        np.random.seed(self.seed)
        # Convert the specified column to datetime if it isn't already
        new_df[self.date_column] = pd.to_datetime(new_df[self.date_column])
        # Generate a random number of seconds for each row (between 0 and max_seconds)
        random_seconds = np.random.randint(0, self.max_seconds + 1, size=len(new_df))
        # Create the new column with the offset added
        new_df[self.new_column] = new_df[self.date_column] + pd.to_timedelta(random_seconds, unit='s')
        return new_df

cast_transformer = CastTypeTransformer(pairColumnTypes={
    'Age' : np.int16,
    'SystolicBP' : np.int16,

```

```

        'DiastolicBP' : np.int16,
        'BS' : np.float32,
        'BodyTemp' : np.float32,
        'HeartRate' : np.int16,
    })
pipeline_dataset = Pipeline([
    ('csv_reader', CsvReader(path='Maternal Health Risk Data Set.csv')),
    ('generate_uuid', GenerateRandomUUID(column='UUID')),
    ('cast_type', cast_transformer),
    ('generate_start_date', GenerateStartDate(start_date='2025-01-01', end_c
    ('random_date_offset', RandomDateOffsetTransformer(date_column='register
])
pipeline_bad_dataset = Pipeline([
    ('delete_records', DeleteRecordsTransformer(p=0.05)),
    ('error_finger', ErrorFingerTransformer(columns='Age', p=0.1)),
    ('missing_values', MissingValuesTransformer(columns=['Age', 'HeartRate', '
    ('duplicate', DuplicateRecordsTransformer(n_duplicates=100)),
    ('c1', cast_transformer),
    (
        'incon',
        AgregarInconsistencia(
            id_column='UUID',
            columns=['Age', 'SystolicBP', 'DiastolicBP', 'BS', 'BodyTemp', 'Heart
            noise_factor=0.2
        )
    ),
    ('c2', cast_transformer),
])
original_dataset = pipeline_dataset.fit_transform(None)
bad_dataset = pipeline_bad_dataset.fit_transform(original_dataset)

```

Explicación Código

Nombre de la función: `pd.merge(left, right, on=None, how='inner', ...)`

Cómo funciona: Combina dos DataFrames de pandas basándose en columnas o índices comunes. Permite especificar el tipo de unión (inner, outer, left, right) para definir cómo se fusionan los datos.

Un ejemplo sencillo de su funcionamiento:

Supongamos que tenemos dos DataFrames:

```
import pandas as pd
```

```
# DataFrame de clientes
```

```
df_clientes = pd.DataFrame({
    'cliente_id': [1, 2, 3],
    'nombre': ['Alice', 'Bob', 'Charlie']
})
```

```

})

# DataFrame de órdenes
df_ordenes = pd.DataFrame({
    'orden_id': [101, 102, 103],
    'cliente_id': [1, 2, 2],
    'producto': ['Laptop', 'Tablet', 'Smartphone']
})

# Realizamos una unión interna basada en la columna 'cliente_id'
df_resultado = pd.merge(df_clientes, df_ordenes, on='cliente_id',
                        how='inner')

print(df_resultado)

```

Nombre de la función: BaseEstimator (de sklearn.base)

Cómo funciona:

BaseEstimator es una clase base en scikit-learn que proporciona funcionalidades comunes para todos los estimadores. Al heredar de BaseEstimator, un modelo obtiene métodos útiles como `get_params()` y `set_params()`, que facilitan la gestión y validación de los hiperparámetros. Esto resulta especialmente ventajoso al integrar estimadores en pipelines y al utilizar técnicas de optimización de parámetros, ya que permite acceder de forma sistemática a la configuración del modelo.

Un ejemplo sencillo de su funcionamiento:

```

from sklearn.base import BaseEstimator

class MiModeloPersonalizado(BaseEstimator):
    def __init__(self, parametro=1):
        self.parametro = parametro

    def fit(self, X, y):
        # Aquí se incluiría la lógica de entrenamiento
        return self

    def predict(self, X):
        # Aquí se incluiría la lógica de predicción
        # Por ejemplo, devolvemos una lista con el valor del
        # parámetro para cada muestra
        return [self.parametro] * len(X)

# Creación del modelo con un parámetro específico
modelo = MiModeloPersonalizado(parametro=42)

# Uso de get_params para obtener los parámetros del modelo
print(modelo.get_params())

```

```
# Salida esperada:  
# {'parametro': 42}
```

Nombre de la función: TransformerMixin (de sklearn.base)

Cómo funciona:

TransformerMixin es una clase base en scikit-learn que facilita la creación de transformadores personalizados. Al heredar de TransformerMixin, un transformador obtiene automáticamente el método `fit_transform()`, el cual combina los métodos `fit()` y `transform()`. **Un ejemplo sencillo de su funcionamiento:**

```
from sklearn.base import BaseEstimator, TransformerMixin  
  
class MiTransformador(BaseEstimator, TransformerMixin):  
    def __init__(self, factor=1):  
        self.factor = factor  
  
    def fit(self, X, y=None):  
        # En este caso, no se requiere aprendizaje, simplemente  
retornamos self  
        return self  
  
    def transform(self, X):  
        # Aplicamos una transformación sencilla: multiplicamos cada  
elemento por el factor  
        return X * self.factor  
  
# Ejemplo de uso del transformador:  
import numpy as np  
  
# Creamos un array de ejemplo  
X = np.array([1, 2, 3, 4, 5])  
  
# Instanciamos el transformador con un factor de 10  
transformador = MiTransformador(factor=10)  
  
# Ajustamos y transformamos el array  
X_transformado = transformador.fit_transform(X)  
  
print(X_transformado)
```

Nombre de la función: Pipeline (de sklearn.pipeline)

Cómo funciona:

Pipeline permite encadenar múltiples pasos de procesamiento y modelado en un único

flujo de trabajo. Cada paso puede ser un transformador (que implementa `fit` y `transform`) o un estimador (que implementa `fit` y `predict`). Esto facilita la construcción, validación y ajuste de parámetros de modelos, al automatizar la secuencia de transformaciones y predicciones.

Un ejemplo sencillo de su funcionamiento:

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

# Definimos un pipeline con dos pasos:
# 1. Escalado de características con StandardScaler
# 2. Clasificación con LogisticRegression
pipeline = Pipeline([
    ('scaler', StandardScaler()),
    ('clf', LogisticRegression())
])

# Ejemplo de uso con datos de entrenamiento:
import numpy as np

# Datos de ejemplo (características) y etiquetas
X = np.array([[1, 2],
               [3, 4],
               [5, 6],
               [7, 8]])
y = np.array([0, 0, 1, 1])

# Ajustamos el pipeline
pipeline.fit(X, y)

# Realizamos una predicción
predicciones = pipeline.predict(X)
print(predicciones)
```

Nombre de la función: HTML (de IPython.display)

Cómo funciona:

La función `HTML` permite encapsular cadenas de texto en formato HTML para ser renderizadas en entornos que soportan contenido HTML, como Jupyter Notebooks. Esto es útil para mostrar contenido formateado, incluyendo etiquetas HTML, estilos y enlaces.

Un ejemplo sencillo de su funcionamiento:

```
from IPython.display import HTML, display

# Creamos un objeto HTML con contenido formateado
```

```
contenido = HTML("<h1>Hola Mundo</h1><p>Este es un ejemplo de  
contenido HTML renderizado.</p>")
```

```
# Para visualizar el contenido en un entorno compatible, usamos  
display:  
display(contenido)
```

Nombre de la función: display (de IPython.display)

Cómo funciona:

La función `display` permite mostrar objetos renderizables en entornos IPython, como Jupyter Notebooks. Esto incluye la visualización de contenido HTML, imágenes, audio, video, y otros formatos interactivos. Facilita la presentación enriquecida de datos y resultados directamente en la interfaz del notebook.

Un ejemplo sencillo de su funcionamiento:

```
from IPython.display import display, HTML  
  
# Creamos un objeto HTML con contenido formateado  
html_content = HTML("<h2>Ejemplo de Display</h2><p>Mostrando  
contenido HTML con display.</p>")  
  
# Usamos display para renderizar el contenido en el notebook  
display(html_content)
```

Evaluación Dataset

Accuracy

```
In [20]: def calculate_and_display_accuracy(bad_dataset, original_dataset):  
# Fusiona los datasets en base a la columna 'UUID'  
merged_df = pd.merge(  
    left=bad_dataset,  
    right=original_dataset,  
    on='UUID',  
    how='left',  
    suffixes=('_bad', '_original')  
)  
  
# Calcula la precisión comparando columnas con sufijos '_bad' y '_original'  
result_accuracy = {}  
for col in merged_df.columns:  
    if col.endswith('_bad'):  
        col_original = col.replace('_bad', '_original')  
        col_clean = col.replace('_bad', '')  
        result_accuracy[col_clean] = round(  
            (merged_df[col] == merged_df[col_original]).mean() * 10,
```

```

        1
    )

    # Convierte el diccionario a DataFrame y calcula la precisión final
    result_accuracy_df = pd.DataFrame(result_accuracy, index=[0])
    cal_accuracy = round(result_accuracy_df.mean(axis=1)[0], 2)
    # Muestra los elementos usando las funciones especificadas
    showMetric('Accuracy Calcification', cal_accuracy)
    show(result_accuracy_df)

    return cal_accuracy

```

In [21]: `cal_accuracy = calculate_and_display_accuracy(bad_dataset, original_dataset)`

Accuracy Calcification: 8.23

Age	SystolicBP	DiastolicBP	BS	BodyTemp	HeartRate
0	6	7.1	8.7	8.7	6.7

Consistency

```

In [22]: def calculate_and_display_consistency(bad_dataset):
    # Filtrar grupos duplicados en base a 'UUID'
    duplicate_groups = bad_dataset.groupby('UUID').filter(lambda x: len(x) > 1)

    def count_discrepancies(group):
        pivot = group.iloc[0]
        counter = 0
        idx = [pivot['UUID']]
        for _, row in group.iloc[1:].iterrows():
            for col in group.columns:
                if pd.isna(pivot[col]) and pd.isna(row[col]):
                    continue
                if pivot[col] != row[col]:
                    # Si al menos un valor es diferente se considera una discrepancia
                    counter += 1
                    idx.append(row['UUID'])
                    break
        return counter, idx

    count_records_with_discrepancies = 0
    ids = []
    total_records = len(bad_dataset)

    # Agrupar nuevamente por 'UUID' en los grupos filtrados
    for _, group in duplicate_groups.groupby('UUID'):
        c, ids_group = count_discrepancies(group)
        ids += ids_group
        count_records_with_discrepancies += c

```

```
records_with_inconsistencies = bad_dataset[bad_dataset['UUID'].isin(ids)]
records_with_inconsistencies = records_with_inconsistencies.sort_values(

cal_consistency = round((1 - count_records_with_discrepancies / total_re

showMetric('Consistency Calcification', cal_consistency)
showTitle('Records with Inconsistencies')
show(records_with_inconsistencies)
return cal_consistency
```

In [23]: `cal_consistency = calculate_and_display_consistency(bad_dataset)`

Consistency Calcification: 9.06

Records with Inconsistencies

10 ▾ entries per page Search:

	Age	SystolicBP	DiastolicBP	BS	BodyTemp	HeartRate
1057	45	135	68	14.669725	96.547562	
746	50	149	83	17.331665	82.43985	
302	45	141	106	20.220461	90.340858	
1027	58	110	90	18.336313	98	
855	NaN	NaN	85	7.9	98	
1002	NaN	NaN	69	7.9	91.152946	
250	32	116	107	19.960163	86.189651	
1020	32	118	103	18.887747	91.371201	
1012	25	139	80	6.459826	111.324265	
136	27	132	92	6.7	111.947075	

Showing 1 to 10 of 200 entries

« ‹ 1 2 3 4 5 ... 20 › »

Completeness

```
In [24]: def calculate_and_display_completeness(bad_dataset):

    result_not_null = bad_dataset.notnull().sum().to_frame().T
    result_non_null_percent = round((bad_dataset.notnull().sum() / len(bad_dataset)), 2)
    cal_completeness = round(result_non_null_percent.mean(axis=1)[0], 2)

    showMetric('Completeness Calcification', cal_completeness)
    showTitle('Cantidad de Registros NO Nulos')
    show(result_not_null)
    showTitle('Calificaciones por Columna')
    show(result_non_null_percent)

    return cal_completeness
```

```
In [25]: cal_completeness = calculate_and_display_completeness(bad_dataset)
```

Completeness Calcification: 9.18

Cantidad de Registros NO Nulos

Age	SystolicBP	DiastolicBP	BS	BodyTemp	HeartRate	Risk
826	864	1067	1067	836	861	

Calificaciones por Columna

Age	SystolicBP	DiastolicBP	BS	BodyTemp	HeartRate	Risk
7.74	8.1	10	10	7.84	8.07	

Opportunity

```
In [26]: def calculate_and_display_opportunity(bad_dataset):
max_date_to_opp = 60 * 60 * 24 * 7 # una semana
result_opp = (bad_dataset['confirm_date'] - bad_dataset['register_date'])
result_opp_mean = round(result_opp.mean(), 2)
result_opp_mean_days = round(result_opp_mean / (60 * 60 * 24), 2)
days = int(result_opp_mean // (60 * 60 * 24))
hours = int((result_opp_mean % (60 * 60 * 24)) // (60 * 60))
result_opp_mean_days_format = f"{days} días con {hours:02d} hrs"
cal_opportunity = 10 - (result_opp_mean / max_date_to_opp) * 10
showMetric('Calificación de Oportunidad', round(cal_opportunity, 2))
showMetric('Tiempo promedio de Confirmación', result_opp_mean_days_format)
return cal_opportunity
```

```
In [27]: cal_opportunity = calculate_and_display_opportunity(bad_dataset)
```

Calificación de Oportunidad: 7.22

Tiempo promedio de Confirmación: 1 días con 22 hrs

Evaluación General

```
In [28]: calificacion_final = round((
    0.4*cal_accuracy + \
    0.2*cal_consistency + \
    0.1*cal_completeness + \
    0.3*cal_opportunity), 2)
showMetric('Calificación Final', calificacion_final)
```

Calificación Final: 8.19

Ejercicio Extra

Significado de los nombres de las columnas

- Si los nombres de las columnas son descriptivos y tienen más de un carácter, es más fácil interpretar los datos.
- Un *dataset* con nombres como "A", "B", "C" es menos accesible que uno con "Nombre", "Edad", "Salario".

2. Estructura del índice

- Un índice con etiquetas significativas (como nombres de productos o fechas) mejora la organización del *dataset*.
- Si el índice es puramente numérico (0,1,2,...), puede indicar falta de una clave significativa.

3. Legibilidad de los datos

- Se verifica que los valores dentro del *dataset* sean comprensibles, evitando valores problemáticos como "N/A", "NaN", "null" o cadenas vacías.
- Un *dataset* con muchos valores ilegibles puede generar problemas en análisis o visualizaciones.

¿Para qué creamos esta dimensión? **Evaluar la calidad de un conjunto de datos antes de su uso**

- Útil en proyectos de *Machine Learning* o análisis de datos, donde la calidad del *dataset* impacta en los resultados.

Identificar problemas en la estructura del dataset

- Si la accesibilidad es baja, se pueden tomar acciones correctivas como renombrar columnas, mejorar el índice o limpiar los datos.

Comparar diferentes conjuntos de datos

- Ayuda a elegir entre múltiples fuentes de datos con base en su accesibilidad.

Cálculo de la Accesibilidad de Datos

El código evalúa la **accesibilidad de los datos** en un conjunto de datos (*DataFrame* de Pandas) utilizando tres criterios y combinándolos en una métrica final.

1. Porcentaje de columnas con nombres significativos

$$\text{columns_percent} = \left(\frac{\text{Número de columnas con nombres de más de un carácter}}{\text{Total de columnas}} \right)$$

2. Porcentaje de índice significativo

$$\text{index_percent} = \begin{cases} 100, & \text{si el índice no es completamente numérico} \\ 0, & \text{si el índice es completamente numérico} \end{cases}$$

3. Porcentaje de datos legibles

$$\text{data_legibility_percent} = \left(\frac{\text{Número de valores legibles}}{\text{Total de elementos en el dataset}} \right) \times 100$$

Donde los valores legibles son aquellos que son **int**, **float** o **str**, y no pertenecen a la lista de valores problemáticos (['N/A', 'NaN', 'null', '']).

4. Cálculo del puntaje de accesibilidad en escala de 0 a 100

$$\text{accessibility_score_percent} = \frac{\text{columns_percent} + \text{index_percent} + \text{data_legibility_percent}}{3}$$

5. Conversión del puntaje de accesibilidad a una escala de 0 a 10

$$\text{accessibility_score} = \left(\frac{\text{accessibility_score_percent}}{100} \right) \times 10$$

El puntaje final de accesibilidad (`accessibility_score`) representa la calidad de los datos en términos de facilidad de interpretación y estructura, en una escala de **0 a 10**.

```
In [29]: def calculate_and_display_accessibility(bad_dataset):
# Evaluar si las columnas tienen nombres significativos
meaningful_columns = sum([1 for col in bad_dataset.columns if len(col) > 1])
columns_percent = round((meaningful_columns / len(bad_dataset.columns)) * 100, 2)

# Evaluar si el dataset tiene un índice significativo (nombre de índice)
meaningful_index = not bad_dataset.index.to_series().apply(lambda x: x.isnumeric())
index_percent = 100 if meaningful_index else 0

# Evaluar si los datos son legibles (sin valores especiales o basura)
legible_data = bad_dataset.map(lambda x: isinstance(x, (int, float, str)))
total_elements = bad_dataset.size
data_legibility_percent = round((legible_data.sum() / total_elements) * 100, 2)
```

```

# Calcular accesibilidad promedio en una escala de 0 a 100
accessibility_score_percent = round((columns_percent + index_percent + c

# Convertir accesibilidad promedio a una escala de 0 a 10
accessibility_score = round((accessibility_score_percent / 100) * 10, 2)

# Mostrar resultados
showMetric('Accessibility Calculation', accessibility_score)
showTitle('Accesibilidad de las Columnas')
show(pd.DataFrame({'Columnas Significativas (%)': [columns_percent]}))
showTitle('Accesibilidad del Índice')
show(pd.DataFrame({'Índice Significativo (%)': [index_percent]}))
showTitle('Legibilidad de los Datos')
show(pd.DataFrame({'Datos Legibles (%)': [data_legibility_percent]}))

return accessibility_score

```

Explicacion del codigo:

meaningful_columns : Me cuenta cuántas columnas tienen nombres significativos, es decir, nombres con más de un carácter. Para aclarar `bad_dataset.columns` obtiene todos los nombres de las columnas.

columns_percent : Me calcula el porcentaje de columnas con nombres significativos.

meaningful_index : Verifica si el índice del DataFrame contiene valores que no sean puramente numéricos. Veamos que `not ...` invierte el resultado, de modo que `True` significa "índice significativo" y `False` significa "índice no significativo".
`bad_dataset.index.to_series()` convierte el índice en una serie de Pandas. `.apply(lambda x: isinstance(x, (int, float)))` revisa si cada valor del índice es un número.

index_percent : Asigna un puntaje de accesibilidad al índice: Si es significativo (`True`), asigna 100%, si no es significativo (`False`), asigna 0%.

legible_data : Cuenta cuántos valores dentro del DataFrame son legibles. `.sum().sum()` suma los valores legibles.

map() : Aplica una función a cada elemento de un objeto, como un DataFrame o una serie. Ejemplo

```
df = pd.DataFrame({'Valores': [1, 'Hola', 3.5, None]})
```

```
df['Es número?'] = df['Valores'].map(lambda x: isinstance(x, (int, float)))
```

```
print(df)
```

```
In [30]: cal_accesi = calculate_and_display_accessibility(bad_dataset)
```

Accessibility Calculation: 6.0

Accesibilidad de las Columnas

Columnas Significativas (%) 

100

Accesibilidad del Índice

Índice Significativo (%) 

0

Legibilidad de los Datos

Datos Legibles (%) 

80

```
In [31]: calificacion_final = round((
    0.4*cal_accuracy + \
    0.2*cal_consistency + \
    0.1*cal_completeness + \
    0.2*cal_opportunity + \
    0.1*cal_accesi), 2)
showMetric('Calificación Final', calificacion_final)
```

Calificación Final: 8.07